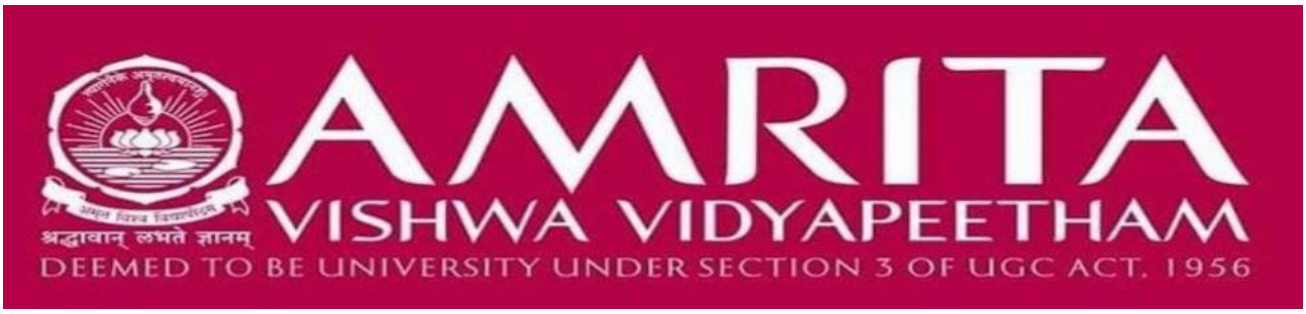


Data structure and algorithms (23CSE203)

LAB MANUAL

Submitted by	
Name	Joshikhaa laasya
YEAR/SEM	Year-2/ sem-3
ROLL.NO	AV. SCU4CSE25117
SECTION	CSE-B

Submitted to	DR. Rajkumar Batchu
DESIGNATION	PROFFESSOR
DEPARTMENT	SCHOOL OF COMPUTING

WEEK	TITLE	PAGE.NO	REMARK
1	A) Recursive program for countdown from n to 1, followed by 'launch'. B) Recursive program to calculate P^n (compound interest growth factor P, years n). C) Recursive program to print first n terms of Fibonacci series. D) Recursive program to search for an employee ID in a list. E) Recursive program to calculate factorial (n!) for parcel arrangement.	4-12	
2	A) Python program to perform Linear Search on given input. B) Python program to perform Binary Search on given input (verify with sorted and unsorted lists).	13-18	
3	A) Python program to implement Bubble Sort (user input). B) Python program to implement Selection Sort (user input). C) Python program to implement Insertion Sort (user input).	19-25	

4	**A) Python program to implement Merge Sort (user input).** **B) Python program to implement Quick Sort (user input).**	**26-30**	
5	**write a python program to implement a single linked list with the below operations** **1) Create linked list** **2) Insert at beginning** **3) Insert at end** **4) Insert at index** **5) Delete by value** **6) Delete first node** **7) Delete last node** **8) Count number of nodes** **9) Display/traverse** **10) Exit**	**31-46**	
6	**A) Implement Stack operations (Push, Pop, Peek, Display) using Array.** **B) Implement Stack operations (Push, Pop, Peek, Display) using Linked List.**	**47-53**	
7	**A) Implement Queue using Arrays.** **B) Implement Queue using Linked List.** **C) Implement Circular Queue using Arrays.** **D) Implement Circular Queue using Linked List.**	**54-63**	
8	**A) Implement Binary Tree with operations using Array.** **B) Implement Binary Tree with operations using Linked List. t**	**64-72**	
9	**Python program to implement Binary Search Tree (BST) operations.**	**73-76**	
10.	**Write a Python program to implement an AVL Tree with all its operations.**	**77-85**	

11	**Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).**	**86-87**	

Week-1

1a.AIM: A rocket launch displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1, followed by “Launch!”.

Code:

launch.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/launch.py (3.14.0)

File Edit Format Run Options Window Help

```
def countdown(n):
    if n>=1:
        print(n)
        countdown(n-1)
    else:
        print("Launch!")

n=int(input("Enter the countdown Number:"))
countdown(n)
```

Output:

Positive case

```
========== RESTART: C:/Users/LENOVO/OneDriv
Enter the countdown Number:10
10
9
8
7
6
5
4
3
2
1
Launch!
```

Negative case

```
========== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/launch.py ====
Enter the countdown Number:-19
Launch!
```

Explanation:

Explanation:

It takes a number as input from the user and stores it in num.

It creates a function named launch().

It starts the countdown from 10 by assigning i = 10.

It prints the value of i as long as num<= i.

It decreases the value of i by 1 after each iteration.

When the loop ends, it prints "Rocket launched" and executes the function using launch().

1b)**AIM:**Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively, write a recursive program to calculate p^n (power) where:

p=principal growth factor, n=no. of years.

Input:

p^n.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/p^n.py (3.14.0)

File Edit Format Run Options Window Help

```
def power(p, n):
    if n == 0:   # base case
        return 1 #joshikhaa laasya
    else:
        return p * power(p, n - 1)

p = int(input("Enter the value of the base: "))
n = int(input("Enter the number of years: "))
result = power(p, n)
print("The growth of an investment is:", result)
```

Output:

Positive case

```
========== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/p^n
Enter the value of the base: 7
Enter the number of years: 9
The growth of an investment is: 40353607
>
```

Negative case

```
========== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/p^n.py ====
Enter the value of the base: -7
Enter the number of years: 8
The growth of an investment is: 5764801
>
```

Explanation:

Explanation:

The program takes the principal growth factor p and number of years n as input.

It defines a recursive function power(p, n).

If n is 0, the function returns 1 (base case).

Otherwise, it returns p * power(p, n-1).

The recursive calls continue until n becomes 0.

The final value of p^n is displayed as the output.

1c) **AIM:** An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function search for an employee ID in a list.

Input:

binarysearch.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysearch.py (3.14.0)

File Edit Format Run Options Window Help

```
def search_employee(emp_list, emp_id, index=0):
    if index >= len(emp_list):
        return False

    if emp_list[index] == emp_id:
        return True

    return search_employee(emp_list, emp_id, index + 1)
#joshikhaa laasya


employees = [101, 102, 103, 104, 105]
emp_id = int(input("Enter employee ID to search: "))

if search_employee(employees, emp_id):
    print("Employee ID found!")
else:
    print("Employee ID not found.")
```

Output:

```
>>>
======= RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysearch.py ======
Enter employee ID to search: 112
Employee ID not found.
>>>
======= RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysearch.py ======
Enter employee ID to search: 102
Employee ID found!
>>>
```

Explanation:

Explanation:

For entering the number as input from user we defined as n

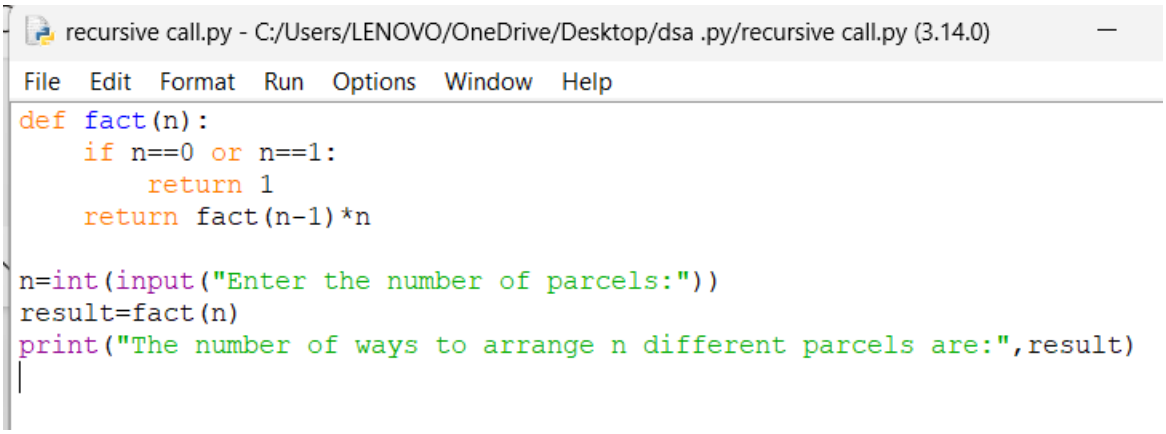
We declared variable for formula.

If input number is less than one then it return “1”

Or it will goes under fibanocci iteration.

1d) **AIM:**In a logistics company, the no of possible ways to arrange n different parcels are calculated using the factorial formula(n!). Develop a recursive program to calculate the factorial of a given number.

Input:



Output:

Positive case

```
====== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/recursive call.py =====
Enter the number of parcels:14
The number of ways to arrange n different parcels are: 87178291200
```

```
====== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/recursive
Enter the number of parcels:-12
Traceback (most recent call last):
  File "C:/Users/LENOVO/OneDrive/Desktop/dsa .py/recursive call.py",
<module>
    result=fact(n)
  File "C:/Users/LENOVO/OneDrive/Desktop/dsa .py/recursive call.py",
fact
```

Explanation;

Explanation:

The program creates a function to search for an employee ID in a list.

It checks one employee ID at a time, starting from the first ID.

If the entered employee ID matches the current ID, it returns True.

If it does not match, the function moves to the next ID using recursion.

If all employee IDs are checked and no match is found, it returns False.

Finally, the program displays "Employee ID found." if the ID exists; otherwise, it displays "Employee ID not found."

1e) **AIM:** Write a recursive python program to print the 1st 'n' terms of Fibonacci series.

Input:

fibanacci.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/fibanacci.py (3.14.0)

File Edit Format Run Options Window Help

```
def fibonacci(n, a=0, b=1, count=0):
    if count == n:
        return
    print(a, end=" ")    # joshikhaa laasya
    fibonacci(n, b, a+b, count+1)


n = int(input("Enter the number of terms: "))
print("Fibonacci series:")
fibonacci(n)
```

output:

Positive case

```
======== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/fibanacci.py ========
Enter the number of terms: 7
Fibonacci series:
0 1 1 2 3 5 8
>>>
```

Negative case:

```
======== RESTART: C:\Users\LENOVO\OneDrive\Desktop\dsa .py\fibanacci.py ========
Enter the number of terms: -9
Fibonacci series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711
 28657 46368 75025 121393 196418 317811 514229 832040 1346269 2178309 3524578 57
02887 9227465 14930352 24157817 39088169 63245986 102334155 165580141 267914296
433494437 701408733 1134903170 1836311903 2971215073 4807526976 7778742049 12586
269025 20365011074 32951280099 53316291173 86267571272 139583862445 225851433717
 365435296162 591286729879 956722026041 1548008755920 2504730781961 405273953788
1 6557470319842 10610209857723 17167680177565 27777890035288 44945570212853 7272
3460248141 117669030460994 190392490709135 308061521170129 498454011879264 80651
5533049393 1304969544928657 2111485077978050 3416454622906707 5527939700884757 8
944394323791464 14472334024676221 23416728348467685 37889062373143906 6130579072
1611591 99194853094755497 160500643816367088 259695496911122585 4201961407274896
73 679891637638612258 1100087778366101931 1779979416004714189 2880067194370816120
 4660046610375530309 7540113804746346429 12200160415121876738 19740274219868223
167 31940434634990099905 51680708854858323072 83621143489848422977 1353018523447
06746049 218922995834555169026 354224848179261915075 573147844013817084101
```

Explanation:

Explanation:

- The program defines a recursive function fact() to calculate the factorial of a number.
- It takes a number as input from the user.
- If the number is 1, the function returns 1 (base case).
- Otherwise, it multiplies the number by the factorial of the previous number (n-1).
- The recursive calls continue until the number becomes 1.
- Finally, the program prints the factorial of the given number.

Error table:

program	error	Correct way

a. Countdown	Forgetting `n-1`
	`No stoppingcondition`
	`Printing Launchfirst`
		countdown(n-1)
		if n == 0
		Print numbers first

b. Power	Using + instead of *
	`No stopping condition`
	`Forgetting n-1`
		p * power(p,n-1)
		if n == 0: return 1
		power(p,n-1)

c. Search	Forgetting to increase index No stopping condition Using wrong index	index + 1 if index == len(arr) Use `arr[index]`

d. Factorial	Using + instead of *
	`No stopping condition`
	`Forgetting n-1`
		n * factorial(n-1)
		if n == 0: return 1
		factorial(n-1)

e. Fibonacci	Using only one recursive call No stopping condition No stopping condition Printing wrong number of terms	fibonacci(n-1) + fibonacci(n-2) if n <= 1 Use `range(n)`

Week-2

2a) **AIM:** Write a python program to implement linear search.

Input:

linearsearch.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/linearsearch.py (3.14.0)

File Edit Format Run Options Window Help

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key: # joshikhaa laasya
            return i   # joshikhaa laasya
    return -1


# Example usage
arr = [10, 25, 30, 45, 50, 75]
key = int(input("Enter the element to search: "))

result = linear_search(arr, key)

if result != -1:
    print(f"Element {key} found at position {result}.")
else:
    print(f"Element {key} not found in the list.")
```

Output:

Positive case

```
======= RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/linearsearch.py ======
Enter the element to search: 25
Element 25 found at position 1.
>
```

```
======= RESTART: C:\Users\LENOVO\OneDrive\Desktop\dsa .py\linearsearch.py
Enter the element to search: -5
Element -5 not found in the list.
```

Explanation:

- In linear search we can append elements in sorted or unsorted way also.

- After placing the target we need to check the all the elements in a order.
- The time complixity is O(n).
- The target element may be found at beginning or last or at middle.

2b)

i)**AIM:** Write a python program to implement binary search with sorted array as an input.

Input:

binary_search.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binary_search.py (3.14.0)

File Edit Format Run Options Window Help

```
def binary_search(arr, key, low, high):
    if low > high:    # Base case: not found
        return -1

    mid = (low + high) // 2 # joshikhaa laasya

    if arr[mid] == key:
        return mid    # Found at index mid
    elif arr[mid] > key:
        return binary_search(arr, key, low, mid - 1)  # Search left half
    else:
        return binary_search(arr, key, mid + 1, high) # Search right half


# Example usage
arr = sorted([10, 25, 30, 45, 50, 75])  # Sorted array
print("Sorted array:", arr)

key = int(input("Enter the element to search: "))
result = binary_search(arr, key, 0, len(arr) - 1)

if result != -1:
    print(f"Element {key} found at position {result}.")
else:
    print(f"Element {key} not found in the array.")
```

output:

Positive case

```
====== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binary_search.py ======
Sorted array: [10, 25, 30, 45, 50, 75]
Enter the element to search: 50
Element 50 found at position 4.
```

Negative case

```
====== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binary_search.py ======
Sorted array: [-50, 10, 25, 30, 45, 75]
Enter the element to search: -50
Element -50 found at position 0.
```

Explanation:

- In linear search we can append elements in sorted or unsorted way also.
- After placing the target we need to check the all the elements in a order.
- The time complixity is O(n).
- The target element may be found at beginning or last or at middle.

ii)**AIM:** Write a python program to implement binary search without sorted array as an input.

Input:

*binarysearch.unsorted.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysearch.unso...

File Edit Format Run Options Window Help

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2 # joshikhaa laasya
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1


# Main program
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

arr.sort()  # Ensure array is sorted
print("Sorted array is:", arr)

key = int(input("Enter the element to search: "))
result = binary_search(arr, key)

if result != -1:
    print("Element is found at the index:", result)
else:
    print("Element is not found")
```

output:

Positive case

```
== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysearch.unsorted.py ==
Enter the number of elements: 6
Enter the elements:
13
15
12
17
25
16
Sorted array is: [12, 13, 15, 16, 17, 25]
```

Negative case

```
Enter the element to search:
== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysea
Enter the number of elements: -4
Enter the elements:
Sorted array is: []
Enter the element to search:
```

Explanation:

- The binary search should always executed in sorted order.
- Either it can be ascending or descending order.
- So the program is in unsorted order so we need to do sorting first later the binary search is executed.

3a) write a python code to implement bubble sort .where the elements are taken from user..

Input:

bubble sort.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/bubble sort.py (3.14.0)

File Edit Format Run Options Window Help

```
# Bubble Sort Implementation in Python

def bubble_sort(arr): # joshikhaa laasya
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

# Main program
n = int(input("Enter the number of elements: "))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

print("Original array:", arr)

bubble_sort(arr)

print("Sorted array:", arr)
```

Output:

Positive case

```
>>>
======= RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/bubble so
Enter the number of elements: 5
Enter the elements:
12
17
13
15
98
Original array: [12, 17, 13, 15, 98]
Sorted array: [12, 13, 15, 17, 98]
>>>
```

Negative case

```
======= RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/bubble sort.py ===
Enter the number of elements: -5
Enter the elements:
Original array: []
Sorted array: []
```

Explanation:

- n stores the number of elements, and an empty list arr is created.
- The first for loop takes n numbers from the user and adds them to arr.
- The second loop controls how many times the sorting process is repeated.
- The inner loop compares two neighboring elements: arr[j] and arr[j+1].
- If the first element is bigger, they are swapped using arr[j], arr[j+1] = arr[j+1], arr[j].
- Finally, print() displays the array in ascending order.

3b) write a python code to implement insertion sort .where the elements are taken from user..

Input:

insertion sort.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/insertion sort.py (3.14.0)

File Edit Format Run Options Window Help

```
# Insertion Sort Implementation in Python

def insertion_sort(arr):
    for i in range(1, len(arr)):  # joshikhaa laasya
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

# Main program
n = int(input("Enter the number of elements: "))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

print("Original array:", arr)

insertion_sort(arr)

print("Sorted array:", arr)
```

Output:

Positive case

```
====== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/insertion sort.py =====
Enter the number of elements: 5
Enter the elements:
12
17
15
13
07
Original array: [12, 17, 15, 13, 7]
Sorted array: [7, 12, 13, 15, 17]
```

Negative case

```
====== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/insertion sort.py =====
Enter the number of elements: 5
Enter the elements:
12
17
15
-13
07
Original array: [12, 17, 15, -13, 7]
Sorted array: [-13, 7, 12, 15, 17]
```

Explanation:

- insertion_sort(arr) takes the array and finds its length using n = len(arr).
- The for loop starts from the second element because the first element is considered already sorted.
- key = arr[j] stores the current element that we want to place in the correct position.
- The while loop moves bigger elements one position to the right.
- arr[j+1] = key places the current element in its correct position.
- Finally, the function returns the sorted array, and print() displays it in ascending order.

3c) write a python code to implement selection sort .where the elements are taken from user..

Input:

selection sort.py - C:\Users\LENOVO\OneDrive\Desktop\dsa .py\selection sort.py (3.14.0)

File Edit Format Run Options Window Help

```
# Selection Sort Implementation in Python

def selection_sort(arr):
    n = len(arr)
    for i in range(n):# joshikhaa laasya
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]

# Main program
n = int(input("Enter the number of elements: "))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

print("Original array:", arr)

selection_sort(arr)

print("Sorted array:", arr)
```

Output:

Positive case

```
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:\Users\LENOVO\OneDrive\Desktop\dsa .py\selection sort.py =====
Enter the number of elements: 5
Enter the elements:
12
13
17
15
07
Original array: [12, 13, 17, 15, 7]
Sorted array: [7, 12, 13, 15, 17]
>>>
```

Negative case

```
>>
====== RESTART: C:\Users\LENOVO\OneDrive\Desktop\dsa .py\selection sort.py =
Enter the number of elements: -5
Enter the elements:
Original array: []
Sorted array: []
>>
```

Explanation;

- n = len(arr) finds the total number of elements in the array.
- The outer for loop selects each position where the smallest element should be placed.
- min_index = i assumes that the current position contains the smallest element.
- The inner for loop compares the remaining elements and finds the smallest element's index.
- arr[i], arr[min_index] = arr[min_index], arr[i] swaps the smallest element with the current element.
- After all iterations, the array is sorted in ascending order and print() displays it.

Error table:

Program	error	correction
Linear search	Using = Wrong loop Forgetting i No result	Use == for i in range(len(arr)) Use arr[i] Print Found / Not Found
Binary search	Array not sorted Wrong middle Wrong left update Wrong right update	Use sorted array mid = (low + high) // 2 low = mid + 1 high = mid – 1

	No loop condition	while low <= high

Binary search(unsorted)	Directly using binary search
	Wrong middle
	Wrong updates
		First use `arr.sort()`
		mid = (low + high) // 2
		`low = mid + 1`, `high = mid - 1`

insertion sort	Wrong starting point Wrong comparison Forgetting shift Forgetting key	Start `i = 1` arr[j] > key arr[j+1] = arr[j] arr[j+1] = key
selection sort	Wrong minimum Wrong comparison Forgetting update Forgetting swap	min = i arr[j] < arr[min] min = j Swap `arr[i]` and `arr[min]`
Bubble sort	Wrong comparison Forgetting swap Wrong loop Wrong outer loop	arr[j] > arr[j+1] Swap adjacent elements range(n-i-1) range(n-1)

WEEK-4

AIM:

Write a python program to implement merge sort .where the elements are taken from the user.

Input:

merge sort.py - C:/Users/CSE-Lab1/Documents/AV.SC.U4CSE25117(PY)/merge sort.py (3.11.4)

File Edit Format Run Options Window Help

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    #divide the array

    mid = len(arr) // 2
    left = merge_sort(arr[:mid]) #AV.SC.U4CSE25117
    right = merge_sort(arr[mid:])
#recursive calls
    return merge(left, right)

#merge the two sorted halves
def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


n = int(input("Enter number of elements: "))

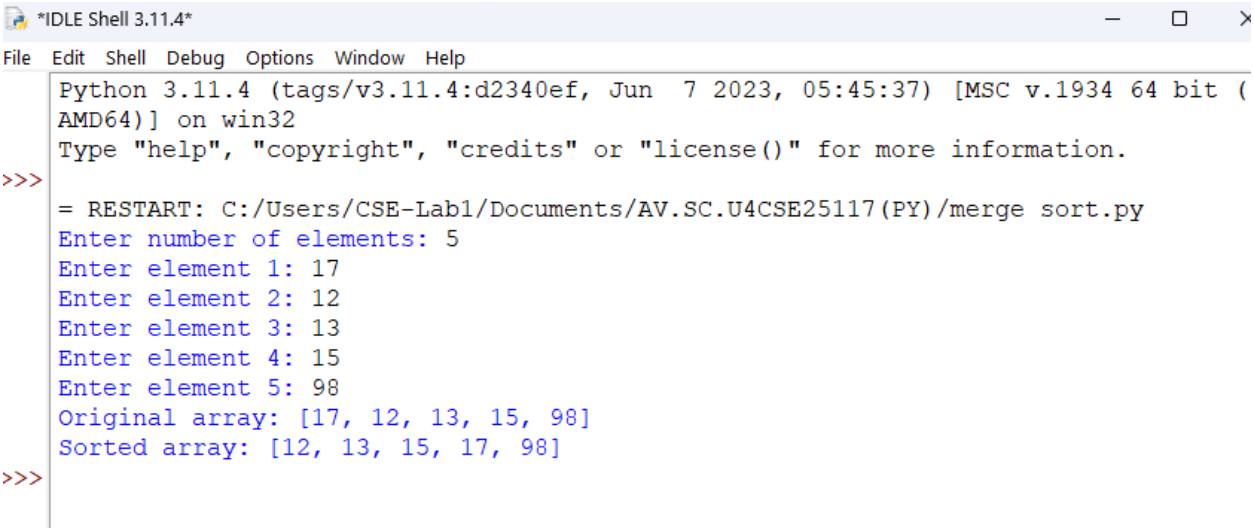
arr = []
for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)

print("Original array:", arr)

arr = merge_sort(arr)
```

Output:

Positive case



Nagetive case

```
= RESTART: C:/Users/CSE-Lab1/Documents/AV.SC.U4CSE25117(PY)/merge sort.py
Enter number of elements: -3
Original array: []
Sorted array: []
```

Explanation;

- if len(arr) <= 1 checks if the array has 0 or 1 element; if yes, it is already sorted.
- mid = len(arr)//2 finds the middle and divides the array into left and right parts.
- merge_sort(left) and merge_sort(right) recursively divide both parts into smaller parts.
- i = j = k = 0 creates indexes to compare elements from the left part, right part, and original array.
- The while loops compare elements and put the smaller element first into the original array.
- Finally, the remaining elements are added, and print() displays the sorted array in ascending order.

b) Write a python program to implement quick sort .where the elements are taken from the user.

Input:

Positive case

quick sort.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/quick sort.py (3.14.0)

File Edit Format Run Options Window Help

```
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)

        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)


def partition(arr, low, high): # joshikhaa laasya
                               # av.sc.u4cse25117
    pivot = arr[high]
    i = low - 1

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]

    return i + 1


n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    x = int(input("Enter element: "))
    arr.append(x)

print("Before sorting:", arr)

quick_sort(arr, 0, n - 1)

print("After sorting:", arr)
```

Output:

```
======== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/quick sort.py =
Enter number of elements: 4
Enter element: 12
Enter element: 15
Enter element: 13
Enter element: 17
Before sorting: [12, 15, 13, 17]
After sorting: [12, 13, 15, 17]
```

Negative case:

```
======== RESTART: C:/Users/LENOVO/OneDrive/Desktop/
Enter number of elements: -9
Before sorting: []
After sorting: []
```

Explanation:

- quick_sort(a, low, high) sorts the elements between the low and high indexes.
- pivot = low selects the first element as the pivot.
- i moves from the left and finds an element greater than the pivot, while j moves from the right and finds an element smaller than or equal to the pivot.
- If i< j, a[i] and a[j] are swapped.
- After the loop, a[j] and the pivot are swapped, putting the pivot in its correct position.
- quick_sort() is called again for the left and right parts, and finally the array is printed in sorted order.

Error table:

programs	Error	correction
Merge sort	Wrong middle Forgetting left side Forgetting right side Wrong comparison Forgetting merge	(low + high) // 2 merge_sort(low, mid) merge_sort(mid+1, high) Compare left and right Merge both parts

Quick sort	Wrong pivot Wrong comparison Forgetting swap Wrong pivot position Forgetting left side Forgetting right side	pivot = arr[high] arr[j] <= pivot Swap elements Put pivot in correct position quick_sort(low, pi-1) quick_sort(pi+1, high)

Week-5

5) write a python program to implement a single linked list with the below operations

11) Create linked list
12) Insert at beginning
13) Insert at end
14) Insert at index
15) Delete by value
16) Delete first node
17) Delete last node
18) Count number of nodes
19) Display/traverse
20) Exit

Input:

single linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/single linked list.py

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # A. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next is not None:
                    temp = temp.next
                temp.next = new_node

    # B. Insert at beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        new_node.next = self.head
        self.head = new_node
```

```
    # C. Insert at end
    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next is not None:
            temp = temp.next

        temp.next = new_node

# D. Insert at index
def insert_index(self):
    data = int(input("Enter data: "))
    index = int(input("Enter index: "))

    new_node = Node(data)

    if index == 0:
        new_node.next = self.head
        self.head = new_node
        return

    temp = self.head

    for i in range(index - 1):
        if temp is None:
            print("Invalid index")
            return
        temp = temp.next

    if temp is None:
        print("Invalid index")
        return

    new_node.next = temp.next
    temp.next = new_node


    # E. Delete by value
    def delete_value(self):
        value = int(input("Enter value to delete: "))

        if self.head is None:
            print("List is empty")
            return

        if self.head.data == value:
            self.head = self.head.next
            return

        temp = self.head

        while temp.next is not None:
            if temp.next.data == value:
                temp.next = temp.next.next
                return
            temp = temp.next

        print("Value not found")
```

```python
# F. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty")
    else:
        self.head = self.head.next
```

```python
# G. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next is None:
        self.head = None
        return

    temp = self.head

    while temp.next.next is not None:
        temp = temp.next

    temp.next = None
```

```python
# H. Count number of nodes
def count(self):
    count = 0
    temp = self.head

    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)
```

```
    # I. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")
```

single linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/single linked list.py

File Edit Format Run Options Window Help

```
# Main program
list1 = SinglyLinkedList()

while True:

    print("\n----- SINGLY LINKED LIST -----")
    print("A. Create linked list")
    print("B. Insert at beginning")
    print("C. Insert at end")
    print("D. Insert at index")
    print("E. Delete by value")
    print("F. Delete first node")
    print("G. Delete last node")
    print("H. Count number of nodes")
    print("I. Display / Traverse")
    print("J. Exit")

    choice = input("Enter your choice: ").upper()

    if choice == 1:
        list1.create()

    elif choice == 2:
        list1.insert_beginning()

    elif choice == 3:
        list1.insert_end()

    elif choice == 4:
        list1.insert_index()

    elif choice == 5:
        list1.delete_value()

    elif choice == 6:
        list1.delete_first()

    elif choice == 7:
        list1.delete_last()

    elif choice == 8:
        list1.count()

    elif choice == 9:
        list1.display()

    elif choice == 10:
```

32°C
Partly cloudy

Output:

```
----- SINGLY LINKED LIST -----
A. Create linked list
B. Insert at beginning
C. Insert at end
D. Insert at index
E. Delete by value
F. Delete first node
G. Delete last node
H. Count number of nodes
I. Display / Traverse
J. Exit
Enter your choice: a
Enter number of nodes: 3
Enter data: 4
Enter data: 6
Enter data: 7

----- SINGLY LINKED LIST -----
A. Create linked list
B. Insert at beginning
C. Insert at end
D. Insert at index
E. Delete by value
F. Delete first node
G. Delete last node
H. Count number of nodes
I. Display / Traverse
J. Exit
Enter your choice: b
Enter data: 12

----- SINGLY LINKED LIST -----
A. Create linked list
B. Insert at beginning
C. Insert at end
D. Insert at index
E. Delete by value
F. Delete first node
G. Delete last node
H. Count number of nodes
I. Display / Traverse
J. Exit
```


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
I. Display / Traverse
J. Exit
Enter your choice: c
Enter data: 17
----- SINGLY LINKED LIST -----
A. Create linked list
B. Insert at beginning
C. Insert at end
D. Insert at index
E. Delete by value
F. Delete first node
G. Delete last node
H. Count number of nodes
I. Display / Traverse
J. Exit
Enter your choice: d
Enter data: 13
Enter index: 2
----- SINGLY LINKED LIST -----
A. Create linked list
B. Insert at beginning
C. Insert at end
D. Insert at index
E. Delete by value
F. Delete first node
G. Delete last node
H. Count number of nodes
I. Display / Traverse
J. Exit
Enter your choice: e
Enter value to delete: 4

```
----- SINGLY LINKED LIST -----
A. Create linked list
B. Insert at beginning
C. Insert at end
D. Insert at index
E. Delete by value
F. Delete first node
G. Delete last node
H. Count number of nodes
I. Display / Traverse
J. Exit
Enter your choice: f

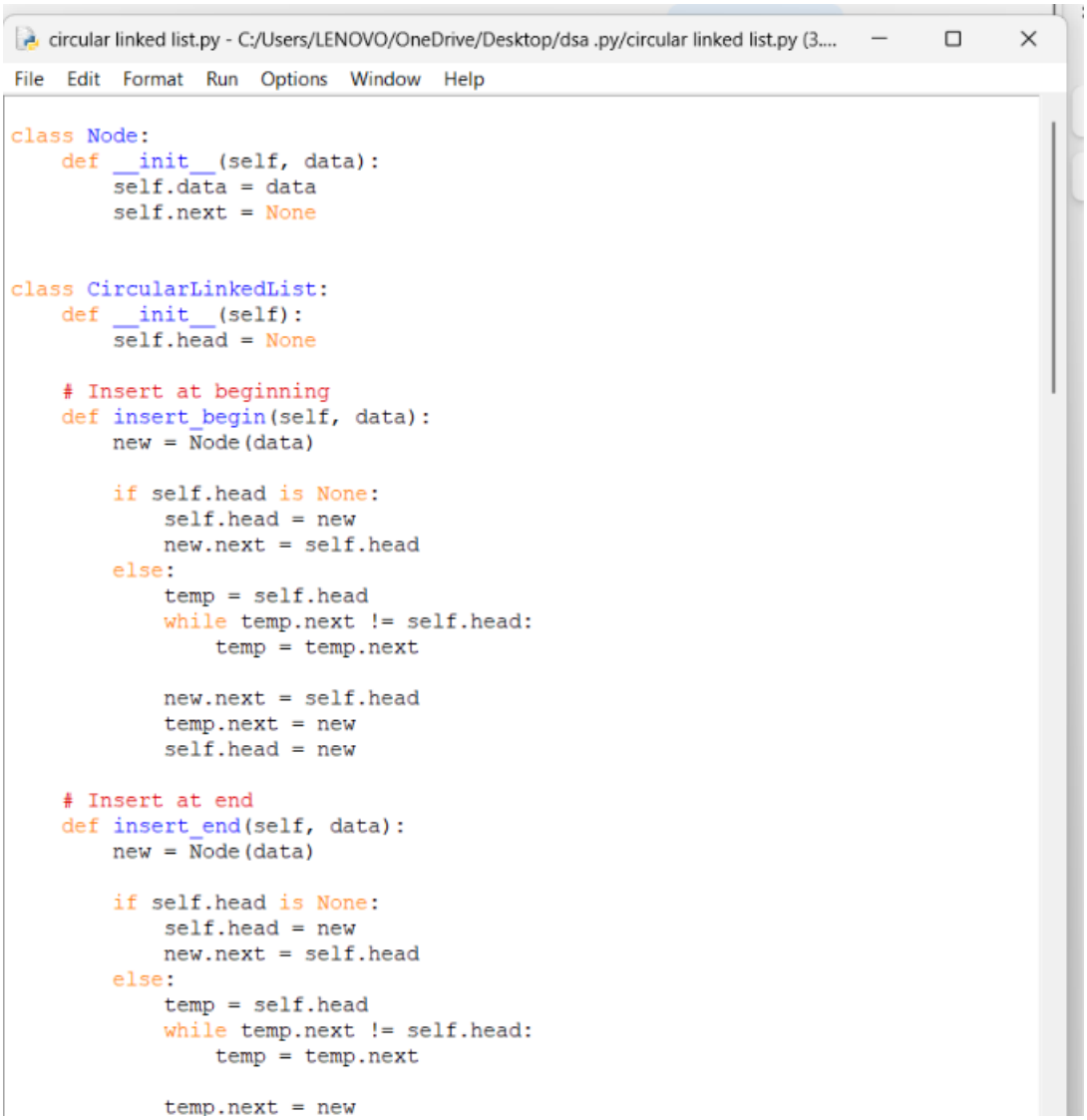
----- SINGLY LINKED LIST -----
A. Create linked list
B. Insert at beginning
C. Insert at end
D. Insert at index
E. Delete by value
F. Delete first node
G. Delete last node
H. Count number of nodes
I. Display / Traverse
J. Exit
Enter your choice: h
Number of nodes: 4

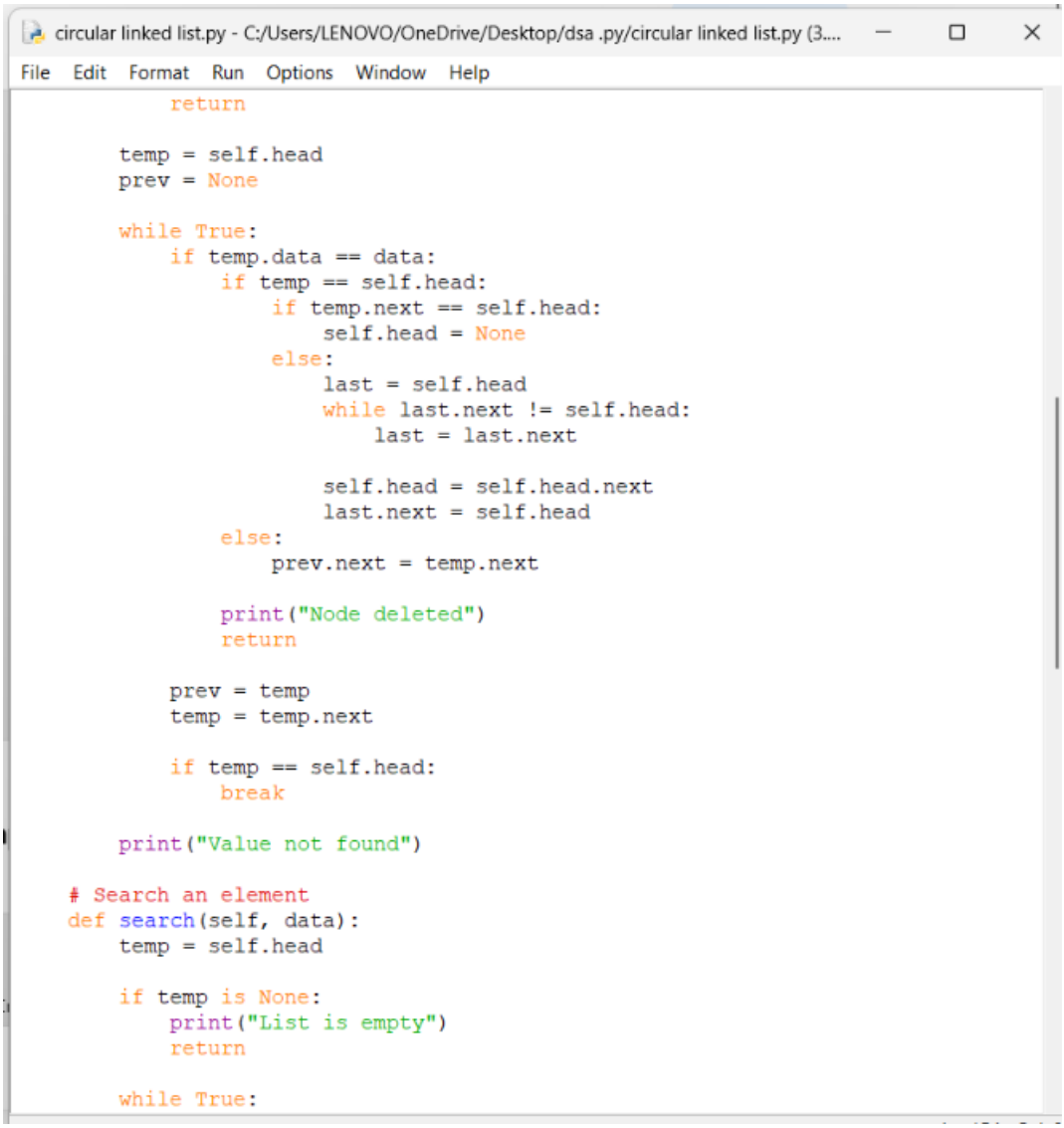
----- SINGLY LINKED LIST -----
A. Create linked list
B. Insert at beginning
C. Insert at end
D. Insert at index
E. Delete by value
F. Delete first node
G. Delete last node
H. Count number of nodes
I. Display / Traverse
J. Exit
Enter your choice: i
13 -> 6 -> 7 -> 17 -> None
13 -> 6 -> 7 -> 17 -> None

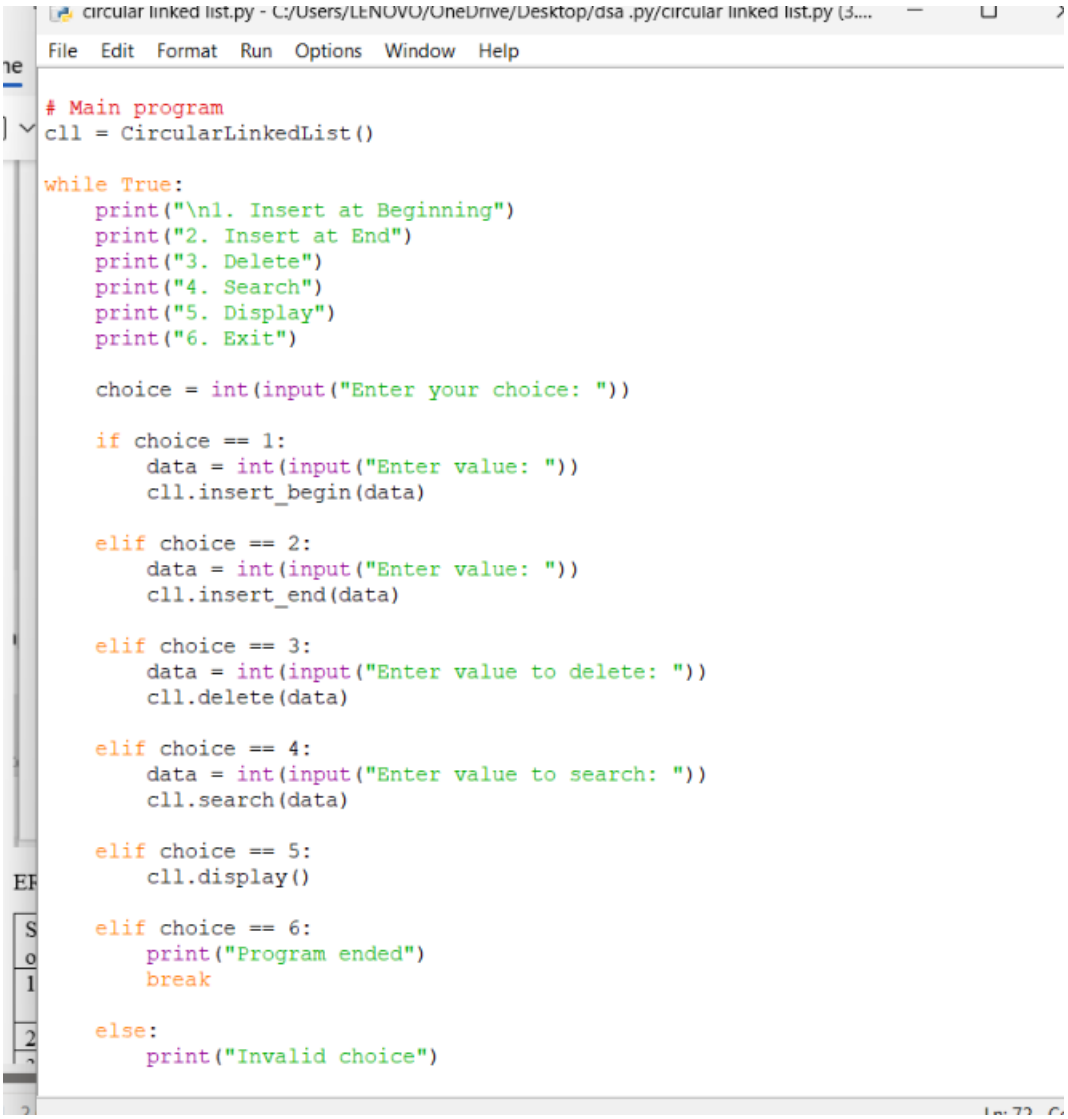
----- SINGLY LINKED LIST -----
A. Create linked list
B. Insert at beginning
C. Insert at end
D. Insert at index
E. Delete by value
F. Delete first node
G. Delete last node
H. Count number of nodes
I. Display / Traverse
J. Exit
Enter your choice: j
Program ended
>>
```

B. Aim:Write a python program to implement a circular linked list which would perform the below operations, by taking an input from the user.

Input:


circular linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/circular linked list.py (3....
File Edit Format Run Options Window Help
class Node:
def __init__(self, data):
self.data = data
self.next = None
class CircularLinkedList:
def __init__(self):
self.head = None
Insert at beginning
def insert_begin(self, data):
new = Node(data)
if self.head is None:
self.head = new
new.next = self.head
else:
temp = self.head
while temp.next != self.head:
temp = temp.next
new.next = self.head
temp.next = new
self.head = new
Insert at end
def insert_end(self, data):
new = Node(data)
if self.head is None:
self.head = new
new.next = self.head
else:
temp = self.head
while temp.next != self.head:
temp = temp.next
temp.next = new


circular linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/circular linked list.py (3....
File Edit Format Run Options Window Help
return
temp = self.head
prev = None
while True:
if temp.data == data:
if temp == self.head:
if temp.next == self.head:
self.head = None
else:
last = self.head
while last.next != self.head:
last = last.next
self.head = self.head.next
last.next = self.head
else:
prev.next = temp.next
print("Node deleted")
return
prev = temp
temp = temp.next
if temp == self.head:
break
print("Value not found")
Search an element
def search(self, data):
temp = self.head
if temp is None:
print("List is empty")
return
while True:


circular linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/circular linked list.py (3....
File Edit Format Run Options Window Help
Main program
cll = CircularLinkedList()
while True:
print("\n1. Insert at Beginning")
print("2. Insert at End")
print("3. Delete")
print("4. Search")
print("5. Display")
print("6. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
data = int(input("Enter value: "))
cll.insert_begin(data)
elif choice == 2:
data = int(input("Enter value: "))
cll.insert_end(data)
elif choice == 3:
data = int(input("Enter value to delete: "))
cll.delete(data)
elif choice == 4:
data = int(input("Enter value to search: "))
cll.search(data)
elif choice == 5:
cll.display()
elif choice == 6:
print("Program ended")
break
else:
print("Invalid choice")

Output:

```
Sorted array: []
>
=== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/circular linked list.py ==

1. Insert at Beginning
2. Insert at End
3. Delete
4. Search
5. Display
6. Exit
Enter your choice: 1
Enter value: 12

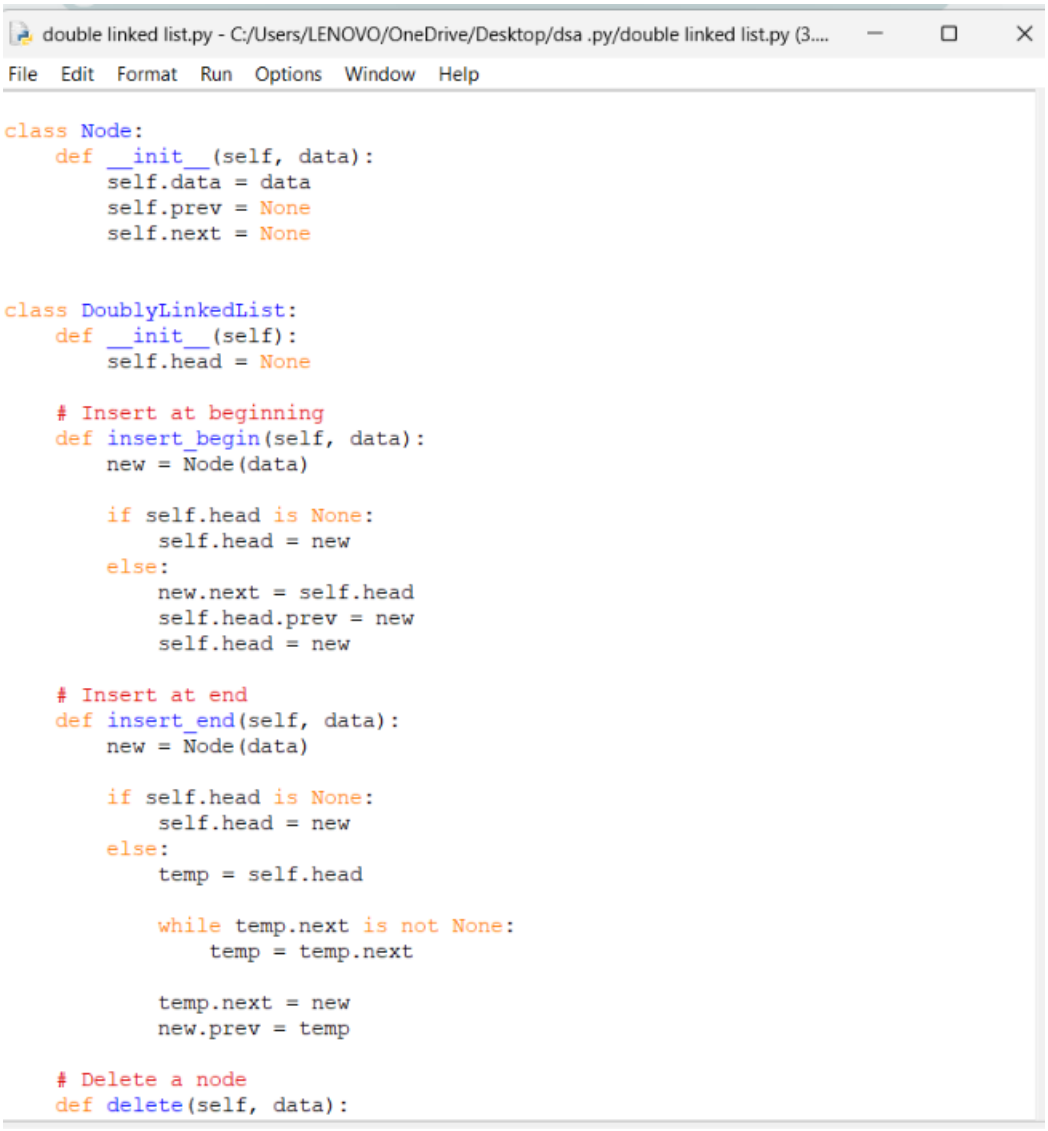
1. Insert at Beginning
2. Insert at End
3. Delete
4. Search
5. Display
6. Exit
Enter your choice: 1
Enter value: 15

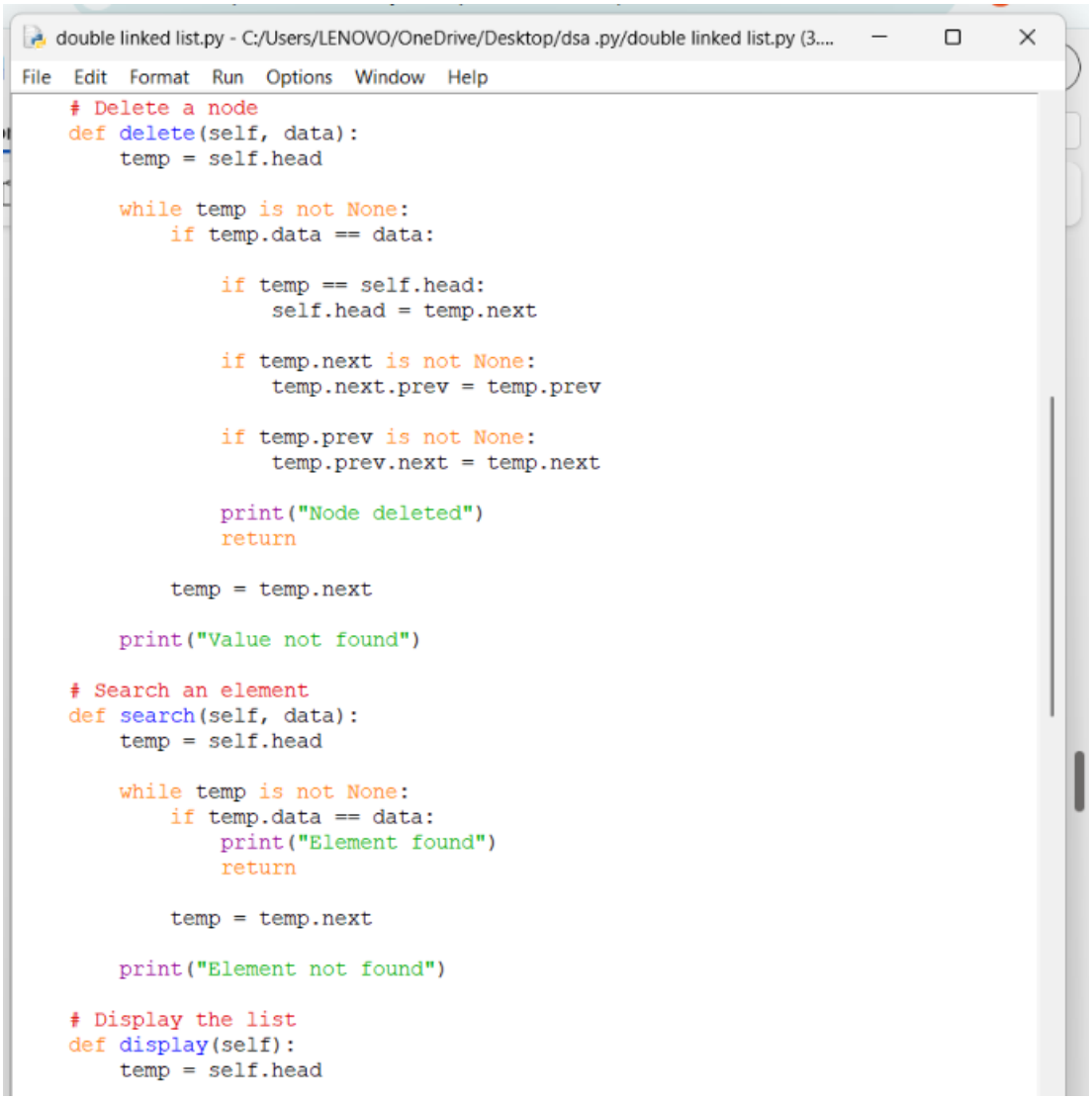
1. Insert at Beginning
2. Insert at End
3. Delete
4. Search
5. Display
6. Exit
Enter your choice: 2
Enter value: 12

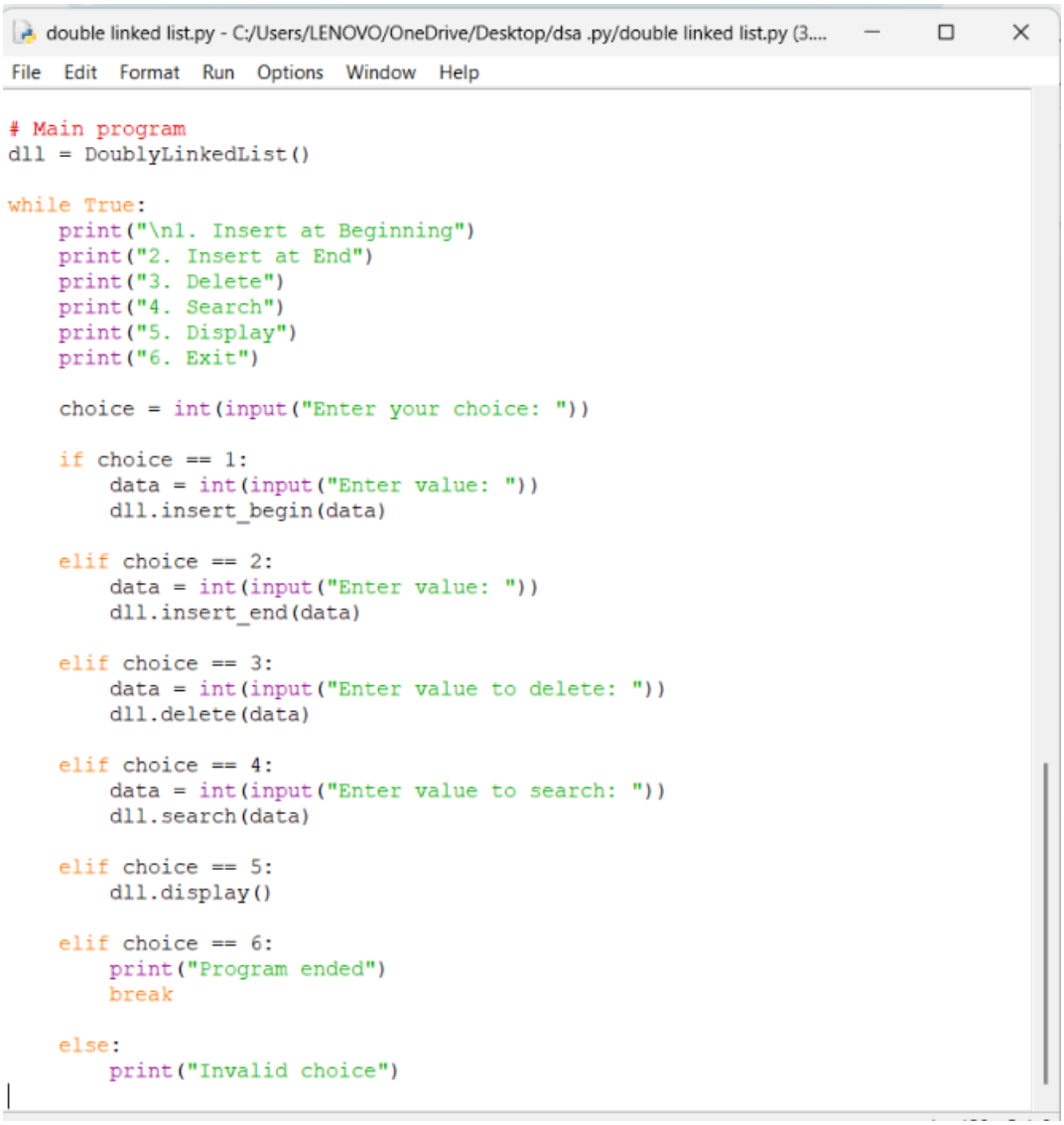
1. Insert at Beginning
2. Insert at End
3. Delete
4. Search
5. Display
6. Exit
Enter your choice:
```

c. Write a python program to implement a doubly linked list which would perform the below operations, by taking an input from the user.

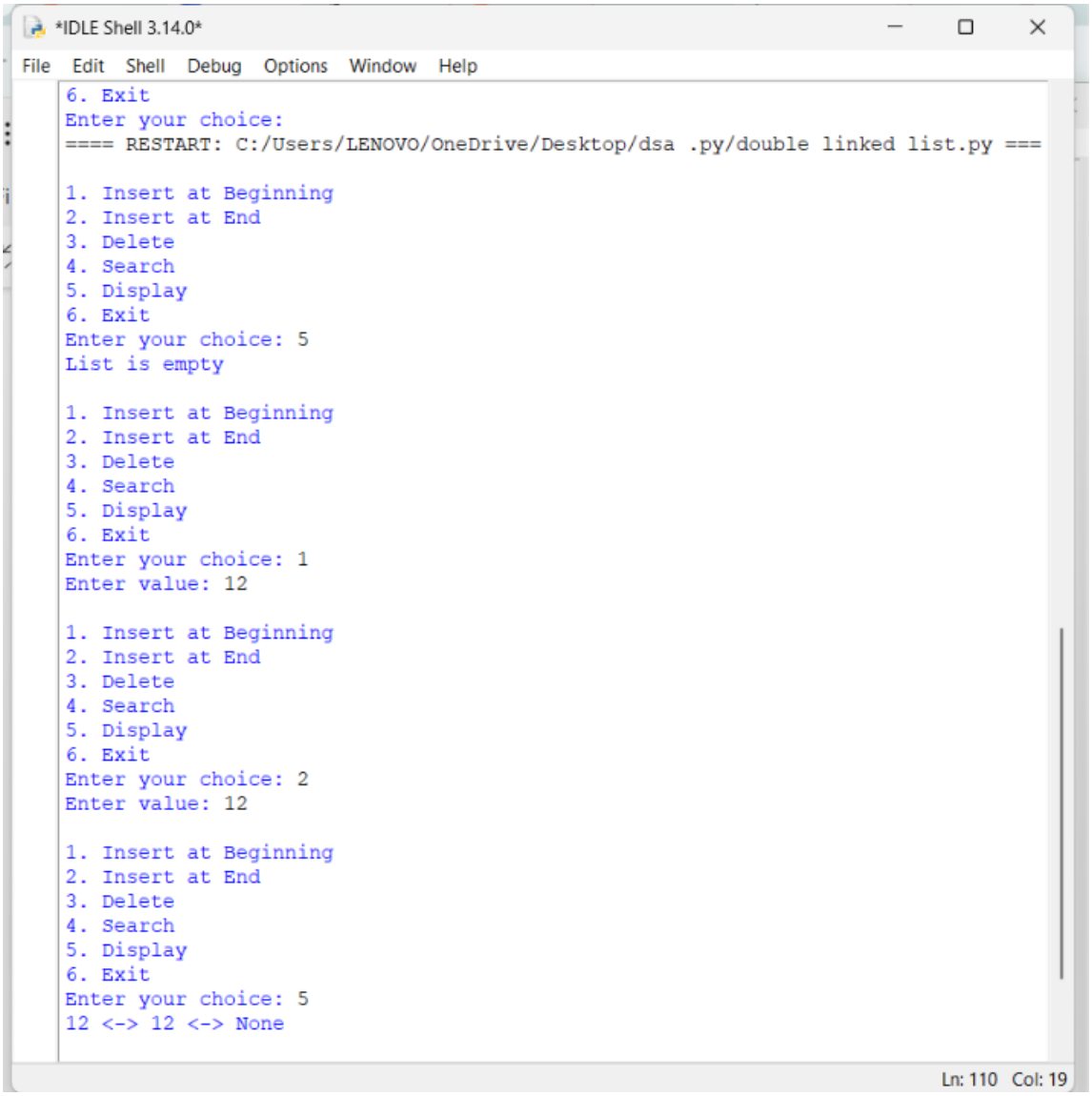
Input:


double linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/double linked list.py (3....
File Edit Format Run Options Window Help
class Node:
def __init__(self, data):
self.data = data
self.prev = None
self.next = None
class DoublyLinkedList:
def __init__(self):
self.head = None
Insert at beginning
def insert_begin(self, data):
new = Node(data)
if self.head is None:
self.head = new
else:
new.next = self.head
self.head.prev = new
self.head = new
Insert at end
def insert_end(self, data):
new = Node(data)
if self.head is None:
self.head = new
else:
temp = self.head
while temp.next is not None:
temp = temp.next
temp.next = new
new.prev = temp
Delete a node
def delete(self, data):


double linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/double linked list.py (3....
File Edit Format Run Options Window Help
Delete a node
def delete(self, data):
temp = self.head
while temp is not None:
if temp.data == data:
if temp == self.head:
self.head = temp.next
if temp.next is not None:
temp.next.prev = temp.prev
if temp.prev is not None:
temp.prev.next = temp.next
print("Node deleted")
return
temp = temp.next
print("Value not found")
Search an element
def search(self, data):
temp = self.head
while temp is not None:
if temp.data == data:
print("Element found")
return
temp = temp.next
print("Element not found")
Display the list
def display(self):
temp = self.head


double linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/double linked list.py (3....
File Edit Format Run Options Window Help
Main program
dll = DoublyLinkedList()
while True:
print("\n1. Insert at Beginning")
print("2. Insert at End")
print("3. Delete")
print("4. Search")
print("5. Display")
print("6. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
data = int(input("Enter value: "))
dll.insert_begin(data)
elif choice == 2:
data = int(input("Enter value: "))
dll.insert_end(data)
elif choice == 3:
data = int(input("Enter value to delete: "))
dll.delete(data)
elif choice == 4:
data = int(input("Enter value to search: "))
dll.search(data)
elif choice == 5:
dll.display()
elif choice == 6:
print("Program ended")
break
else:
print("Invalid choice")

Output:



Explanation:

Singly Linked List (SLL):

A Singly Linked List is a linear data structure in which elements are stored in the form of nodes. Each node consists of two parts: data and a next pointer. The data part stores the value, and the next pointer stores the address of the next node. Each node is connected to the next node in sequence. The first node is called the head, and the last node points to `None`, indicating the end of the list. In a singly linked list, traversal is possible only in the forward direction. Insertion, deletion, searching and traversal are the basic operations performed on a singly linked list. It is used in implementing stacks, queues and memory management.

Doubly Linked List (DLL)

A Doubly Linked List is a linear data structure in which each node consists of three parts: previous pointer, data and next pointer. The data part stores the value, the previous pointer stores the address of the previous node, and the next pointer stores the address of the next node. The first node's previous pointer points to `None`, and the last node's next pointer points to `None`. Unlike a singly linked list, a doubly linked list allows traversal in both forward and backward directions. Its basic operations include insertion, deletion, searching and traversal. It is used in browser forward and backward navigation, undo and redo operations, and music playlists.

Circular Linked List (CLL)

A Circular Linked List is a linked list in which the last node is connected back to the first node instead of pointing to `None`. Each node consists of two parts: data and a next pointer. The data part stores the value, and the next pointer stores the address of the next node. The last node points to the head, forming a circular structure. Unlike a normal singly linked list, there is no `None` at the end of the list. Traversal continues until the starting node is reached again. Its basic operations include insertion, deletion, searching and traversal. It is used in Round Robin CPU scheduling, circular playlists and turn-based games.

ERROR TABLE:

Sno:	error	reason	correction
1	Value error	Entering a non-integer value for menu choice or data	Enter a valid integer.
2	Invalid choies	Selecting a number outside 1 to 5	Enter a choice between 1 and 5
3	List is empty	Trying to delete a node from an empty list	Insert a node before deleting.
4	Element not found	Searching for or deleting a value that does not exist	Enter a value present in the linked list.
5	Empty list display	Displaying the list before inserting any node	Insert elements before displaying.
6	Attribute error	Accessing an attribute of a `None` object due to incorrect pointer handling	Check whether the node is `None` before accessing its attributes.

Week-6

Aim: 6a. Write a python program to implement stack operation such as push,pop,peek,display using an array

Input:

stack using array.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/stack using array.py (3.14.0)

File Edit Format Run Options Window Help

```
stack = []
size = int(input("Enter stack size: "))

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        if len(stack) == size:
            print("Stack Overflow")
        else:
            value = int(input("Enter value: "))
            stack.append(value)
            print("Value pushed")

    elif choice == 2:
        if len(stack) == 0:
            print("Stack Underflow")
        else:
            print("Popped:", stack.pop())

    elif choice == 3:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Top element:", stack[-1])

    elif choice == 4:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Stack:", stack)

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

Output:

```
>>>
==== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/stack using array.py ====
Enter stack size: 5

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 5
Value pushed

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 4
Value pushed

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 2
Popped: 4

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 4
Stack: [5]

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 5
>>>
```

Explanation:

Explanation:

- Stack is a linear data structure.it follows the LIFO(last in first out).
- Elements are stored in the array
- Push is used to insert an element. The element is added at the top. If the stack is full, Stack Overflow occurs.
- Pop is used to remove an element.
- The top element is removed.
- top is decreased. If the stack is empty, Stack Underflow occurs.
- Peek is used to view the top element.
- It does not remove the element.
- Display is used to show all elements.

- Elements are displayed from top to bottom.

6b) Write a python program to implement stack operation such as push,pop,peek,display using an linked list

Input:

stack using linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/stack using linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    # Push
    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print("Value pushed")

    # Pop
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped:", self.top.data)
            self.top = self.top.next

    # Peek
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    # Display
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top

            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next

            print("None")
```

```
stack = Stack()

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        stack.push(value)

    elif choice == 2:
        stack.pop()

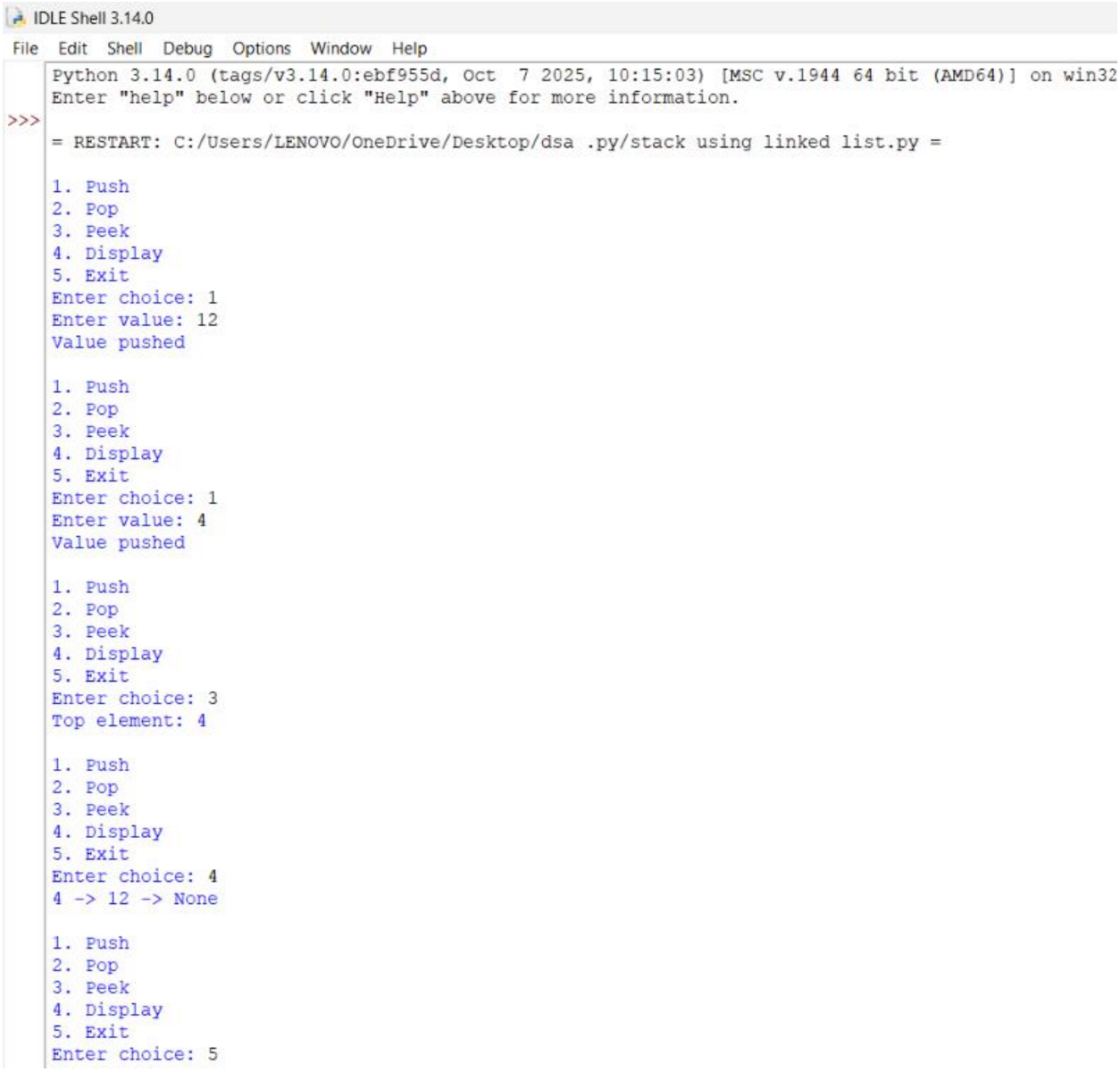
    elif choice == 3:
        stack.peek()

    elif choice == 4:
        stack.display()

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

Output:



Explanation:

- A stack using linked list is a linear data structure that follows the LIFO (Last In, First Out) principle.
- In a linked-list stack, each element is stored in a node, and each node contains data and a link to the next node.
- A variable called top is used to point to the first node, which represents the top of the stack.
- Push operation is used to insert a new element. A new node is created and added at the top of the stack.

Pop operation is used to remove an element. The node pointed to by top is removed and top moves to the next node.

- Peek operation is used to view the top element without removing it from the stack.
- Display operation is used to traverse the linked list from top to None and display all the elements.
- A linked-list stack has dynamic size, so memory is allocated as needed. Stack Underflow occurs when we try to remove an element from an empty stack.

Error table:

error	reason	Correction
Index error	Trying to pop from an empty stack	Check if stack is empty
Stack Overflow	Trying to push when stack is full	Check stack size before push
Stack Underflow	Trying to pop when stack is empty	Check `top` before pop
AttributeError	Using wrong node variable or attribute	Check `data` and `next`
NameError	Variable/class name is wrong	Check spelling
TypeError	Wrong type of value is given	Enter correct data type
IndentationError	Incorrect spacing/indentation	Align the code properly
SyntaxError	Missing `:`, bracket, etc.	Check syntax

Aim: 7a: write a python program to implement queue and its operations such as enqueue ,dequeue,peek,display with a memu driven approch.implement using array and linked list data structure

Input:

queue using linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/queue using linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
queue = []
size = int(input("Enter queue size: "))

while True:
    print("\n--- QUEUE ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        if len(queue) == size:
            print("Queue Overflow")
        else:
            value = int(input("Enter value: "))
            queue.append(value)
            print("Value inserted")

    elif choice == 2:
        if len(queue) == 0:
            print("Queue Underflow")
        else:
            print("Deleted:", queue.pop(0))

    elif choice == 3:
        if len(queue) == 0:
            print("Queue is empty")
        else:
            print("Front element:", queue[0])

    elif choice == 4:
        if len(queue) == 0:
            print("Queue is empty")
        else:
            print("Queue:", queue)

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

Output

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/queue using linked list.py =
Enter queue size: 5

--- QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 18
Value inserted

--- QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 2
Deleted: 18

--- QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 12
Value inserted

--- QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 3
Front element: 12

--- QUEUE ---
1. Enqueue
2. Dequeue
```

```
4. Display
5. Exit
Enter choice: 3
Front element: 12

--- QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 4
Queue: [12]

--- QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 5
>>>
```

Explanation:

- Queue is the linear data structure. It follows first in first out rule.
- In queue for insertion we enqueue and for deletion we use dequeue.
- Before inserting an element we check whether the queue is filled or if it is filled then it is queue overflow.
- Before deletion we will check whether any element is present in the queue or not if any element is not present then it is queue underflow.
- For seeking the top most element we use peek.
- And for inserting, deleting, peek and for display we use menu driven system.
- Users choice will be considerable.

7b)write a python program to implement queue and its operations such as enqueue ,dequeue,peek,display with a memu driven approch.implement using array and with circular queue

Input:

queue using circular queqe.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/queue using circular queqe.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue
    def enqueue(self, data):
        new_node = Node(data)

        if self.rear is None:
            self.front = new_node
            self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print("Value inserted")

    # Dequeue
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            print("Deleted:", self.front.data)
            self.front = self.front.next

            if self.front is None:
                self.rear = None

    # Peek
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    # Display
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
```

queue using circular queqe.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/queue using circular queqe.py (3.14.0)

File Edit Format Run Options Window Help

```
            print("Front element:", self.front.data)

    # Display
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            temp = self.front

            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next

            print("None")


q = Queue()

while True:
    print("\n--- QUEUE USING LINKED LIST ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        q.enqueue(value)

    elif choice == 2:
        q.dequeue()

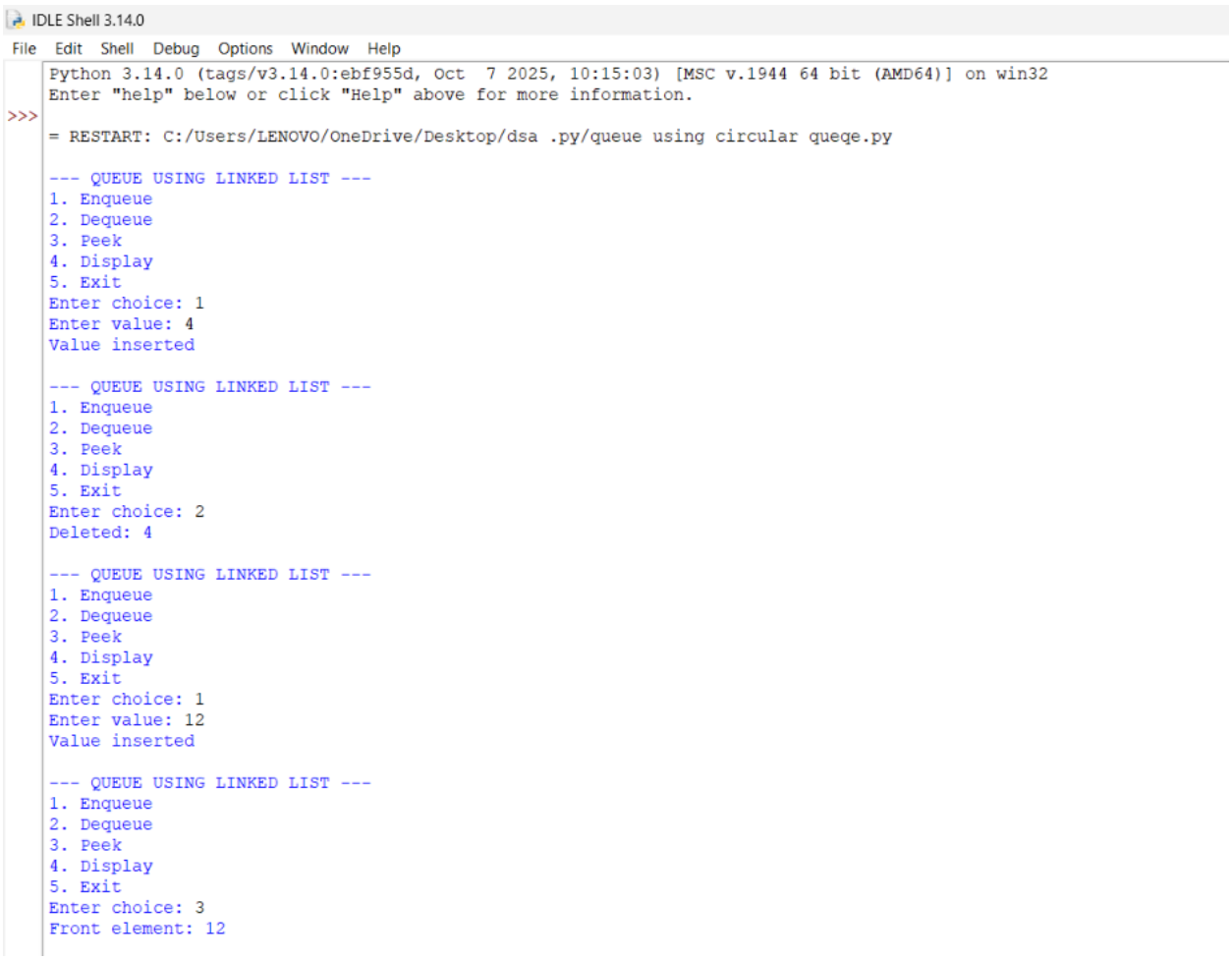
    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

Output;


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/queue using circular queqe.py
--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 4
Value inserted
--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 2
Deleted: 4
--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 12
Value inserted
--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 3
Front element: 12

expla



Explanation:

- Queue is the linear data structure. It follows first in first out rule.
- In queue for insertion we enqueue and for deletion we use dequeue.
- Before inserting an element we check whether the queue is filled or if it is filled then it is queue overflow.
- Before deletion we will check whether any element is present in the queue or not if any element is not present then it is queue underflow.
- For seeking the top most element we use peek.

- And for inserting, deleting, peek and for display we use menu driven system.
- Users choice will be considerable.

Implemented queue:

- Each node stores data and a link (next) to the next node.
- Has two pointers: front (start of queue) and rear (end of queue).
- Creates a new node.
- If queue is empty → both front and rear point to this node.
- Otherwise → attaches new node at the end and moves rear.
- Removes the node at front.
- Moves front to the next node.
- If queue becomes empty → rear also set to None.
- Shows the data at front without removing it.
- If empty → prints *Queue is empty*.
- Starts from front and prints each node until the end.
- Shows elements in order with arrows (->).
- Enqueue works fine on empty queue.
- Dequeue, Peek, Display all check if queue is empty before acting.
- Unlike array-based queue, linked list queue grows/shrinks as needed.
- No fixed size or overflow problem (only limited by memory).
- User chooses operations: Enqueue, Dequeue, Peek, Display, Exit.
- Runs in a loop until Exit is chosen.

Error table:

Error	Reason	Correction
Queue Overflow	Queue is full	Check before enqueue
Queue Underflow	Queue is empty	Check before dequeue
IndexError	Wrong index/accessing empty queue	Check before dequeue
NameError	Wrong variable name	Check spelling
TypeError	Wrong data type	Enter correct value
AttributeError	Wrong node/attribute	Check `data` and `next`
IndentationError	Wrong spacing	Correct indentation
SyntaxError	Wrong Python syntax	Check `:`, brackets, etc.

WEEK-8

8a.write a python program to implement a binary tree with its operations using array..

Input:

binarytree.py - C:/Users/CSE-Lab1/Documents/AV.SC.U4CSE25117(PY)/binarytree.py (3.11.4)

File Edit Format Run Options Window Help

```
    #set left child
    def set_left(self, parent_index, data):
        child_index = 2 * parent_index + 1
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("index out of range!")

    #set right child
    def set_right(self, parent_index, data):
        child_index = 2 * parent_index + 2
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("index out of range")

    #pre order traversal
    def preorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        print(self.tree[index], end=" ")
        self.preorder(2 * index + 1)
        self.preorder(2 * index + 2)

    #inorder traversal
    def inorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        self.inorder(2 * index + 1)
        print(self.tree[index], end=" ")
        self.inorder(2 * index + 2)

    #post order traversal
    def postorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        self.postorder(2 * index + 1)
        self.postorder(2 * index + 2)
        print(self.tree[index], end=" ")

    #level-order traversal
    def level_order(self):
        for value in self.tree:
            if value is not None:
                print(value, end=" ")

    #display array
    def display(self):
```

binarytree.py - C:/Users/CSE-Lab1/Documents/AV.SC.U4CSE25117(PY)/binarytree.py (3.11.4)

File Edit Format Run Options Window Help

```
        self.inorder(2 * index + 1)
        print(self.tree[index], end=" ")
        self.inorder(2 * index + 2)

    #post order traversal
    def postorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        self.postorder(2 * index + 1)
        self.postorder(2 * index + 2)
        print(self.tree[index], end=" ")

    #level-order traversal
    def level_order(self):
        for value in self.tree:
            if value is not None:
                print(value, end=" ")

    #display array
    def display(self):
        print("\narray representation")
        for i in range(self.size):
            print(f"index {i}: {self.tree[i]}")

#main program
tree = binarytreearray(7)
tree.set_root('A')
tree.set_left(0, 'B')
tree.set_right(0, 'C')

tree.set_left(1, 'D')
tree.set_right(1, 'E')
tree.set_left(2, 'F')

tree.display()

print("\npreorder: ", end="")
tree.preorder()

print("\ninorder: ", end="")
tree.inorder()

print("\npostorder: ", end="")
tree.postorder()

print("\nlevel_order: ", end="")
tree.level_order()
```

output:

IDLE Shell 3.11.4

File Edit Shell Debug Options Window Help

```
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023, 05:45:37) [MSC v.1934 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/CSE-Lab1/Documents/AV.SC.U4CSE25117(PY)/binarytree.py

array representation
index 0: A
index 1: B
index 2: C
index 3: D
index 4: E
index 5: F
index 6: None

preorder: A B D E C F
inorder: D B E A F C
postorder: D E B F C A
level_order: A B C D E F
```

Explanation:

- The Node class defines each tree element with data, left, and right links.
- The BinaryTree class manages the tree, starting with root = None.
- create_tree() builds a fixed binary tree with nodes A–F arranged in a specific structure.
- Recursive methods (preorder, inorder, postorder) traverse the tree in different orders and print node values.
- level_order() uses a queue to perform breadth-first traversal, visiting nodes level by level.
- The program prints traversal outputs: Preorder → A B D E C F, Inorder → D B E A C F, Postorder → D E B F C A, Level-order → A B C D E F.
- The recursive traversal functions stop when they reach a None node, ensuring the recursion terminates correctly.
- The queue in level_order() is a simple list, and nodes are processed in FIFO order to achieve breadth-first traversal.
- The print(node.data,end=" ") statements ensure all outputs appear on the same line with spaces instead of newlines.
- The tree structure is hardcoded in create_tree(); it doesn't take user input, so the same tree is always built.
- Each traversal method demonstrates a different way of exploring the same tree, useful for understanding tree algorithms.
- The program's final output shows how traversal orders differ, helping visualize the relationship between tree structure and traversal logic.

Error table:

sno	error	correction
1	Root already exists	Insert the root only if the tree is empty.

2	Parent node not found	Check whether the parent exists in the array.
3	Left child already exists	Insert only if the left child position is empty.
4	Right child already exists	Insert only if the right child position is empty.
5	Array capacity exceeded	i ncrease the array size or use a larger array

8b) Write a python program to implement a binary tree with its operations using linked list

Input:

binary search using linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binary search using linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
# BINARY TREE USING LINKED REPRESENTATION

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


# BINARY TREE CLASS
class BinaryTree:
    def __init__(self):
        self.root = None

    # Create the binary tree
    def create_tree(self):
        self.root = Node(input("Enter root node: "))

        # Left and right children of root
        self.root.left = Node(input("Enter left child of root: "))
        self.root.right = Node(input("Enter right child of root: "))

        # Children of left child
        self.root.left.left = Node(input("Enter left child of left node: "))
        self.root.left.right = Node(input("Enter right child of left node: "))

        # Right child of root
        self.root.right.right = Node(input("Enter right child of right node: "))

    # Preorder Traversal: Root, Left, Right
    def preorder(self, node):
        if node is None:
            return

        print(node.data, end=" ")
        self.preorder(node.left)
        self.preorder(node.right)

    # Inorder Traversal: Left, Root, Right
    def inorder(self, node):
        if node is None:
            return

        self.inorder(node.left)
        print(node.data, end=" ")
        self.inorder(node.right)
```

binary search using linked list.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binary search using linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
    def postorder(self, node):
        if node is None:
            return

        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data, end=" ")

    # Level-order Traversal
    def level_order(self):
        if self.root is None:
            return

        queue = [self.root]

        while queue:
            node = queue.pop(0)
            print(node.data, end=" ")

            if node.left is not None:
                queue.append(node.left)

            if node.right is not None:
                queue.append(node.right)


# MAIN PROGRAM
tree = BinaryTree()

# Create the binary tree
tree.create_tree()

# Traversals
print("\nPreorder: ", end="")
tree.preorder(tree.root)

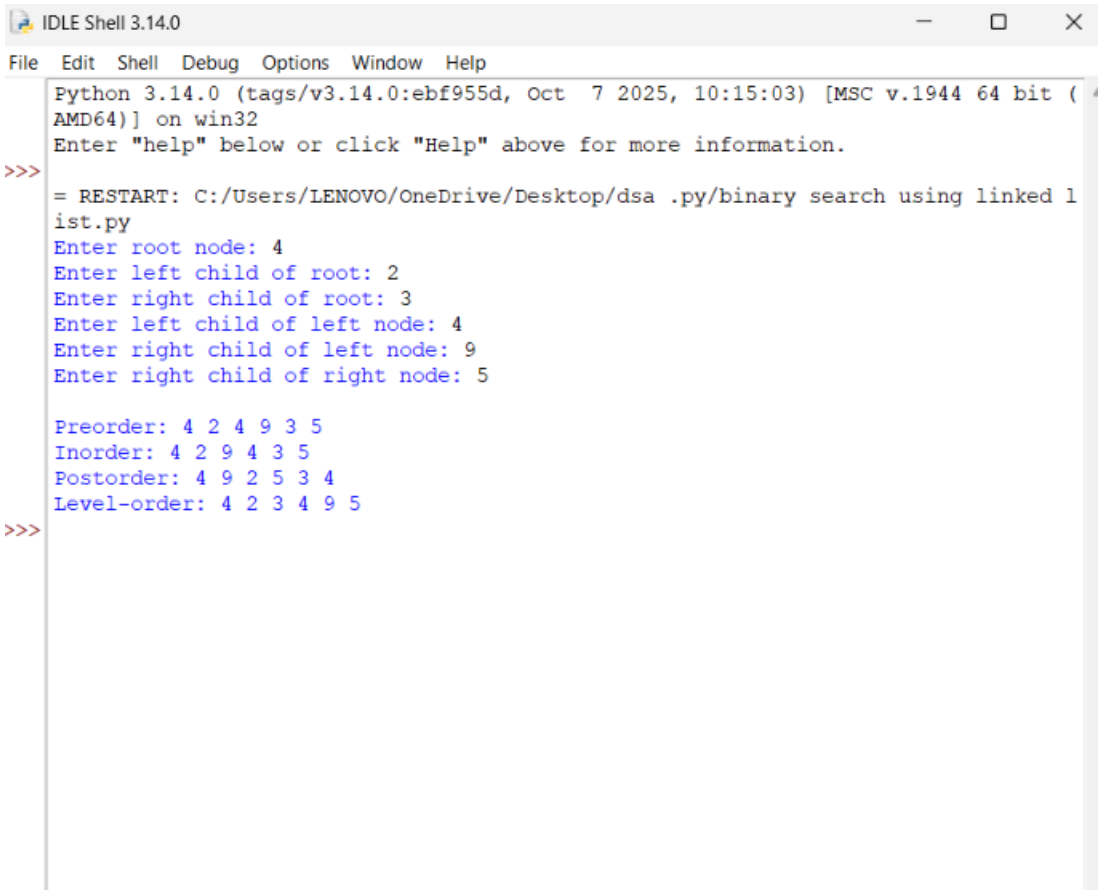
print("\nInorder: ", end="")
tree.inorder(tree.root)

print("\nPostorder: ", end="")
tree.postorder(tree.root)

print("\nLevel-order: ", end="")
tree.level_order()

print()
```

output:



Explanation:

- The code creates a binary tree using an array instead of linked nodes.
- The root is stored at index 0, and children are calculated using formulas: left child → 2*parent_index+1, right child → 2*parent_index+2.
- If the calculated index is within the array size, the child is added; otherwise, it prints "Index out of range!".
- Preorder traversal prints the current node first, then left child, then right child.
- Inorder traversal prints the left child, then the current node, then the right child.
- Postorder traversal prints children first, then the current node.

- Level order traversal simply loops through the array and prints all non-None values.
- The display() method shows the array representation with index numbers and stored values.
- The tree is built with nodes A–F, and then all traversals are executed to show different ways of visiting nodes.
- This demonstrates how binary trees can be represented and traversed using arrays instead of node pointers.

Errortable:

S.no	Errors	Rectified
1.	Printing none values in level order	Rectified
2.	Root is not set before traversal	Rectified by calling set_root() first.

Week-9

9. write a python program to implememt a binary search tree with its operations and traversals..

Input:



```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root

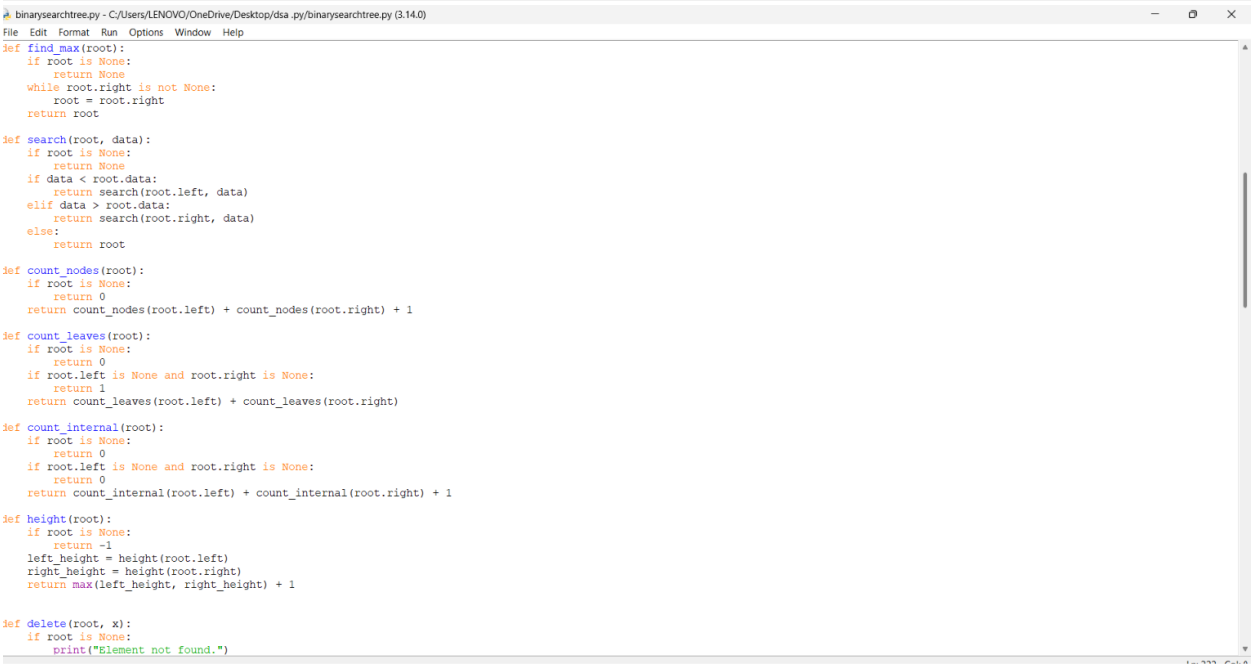

def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
```



```
def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1


def delete(root, x):
    if root is None:
        print("Element not found.")
```

binarysearchtree.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysearchtree.py (3.14.0)

File Edit Format Run Options Window Help

```
def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)


def level_order(root):
    h = height(root)
    for level in range(h):
        print_level(root, level)


# ====================================================
# MAIN PROGRAM
# ====================================================

root = None
while True:
```

binarysearchtree.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysearchtree.py (3.14.0)

File Edit Format Run Options Window Help

```
# ====================================================
# MAIN PROGRAM
# ====================================================

root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
    print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            preorder(root)
            print()
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()
    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
```

```
binarysearchtree.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysearchtree.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
            print("Tree is empty.")
        else:
            postorder(root)
            print()
    elif choice == 5:
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 9:
        print("Number of nodes:", count_nodes(root))
    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

output:

```
*IDLE Shell 3.14.0*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysearchtree.py ====


1. Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 4


1. Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice:
===== RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/binarysearchtree.py ====


******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 12
Element inserted successfully.


******** BINARY SEARCH TREE ********
```

Explanation:

The Node class is used to create a node containing data and two pointers, left and right, which point to the left and right child nodes.

2. The insert() function adds a new node to the Binary Search Tree. Smaller values are placed on the left side, and larger values are placed on the right side.

3. The search() function checks whether the given element exists in the tree. It compares the element with the current node and searches the left or right subtree.

4. The delete() function removes a node from the tree. It handles nodes with no children, one child, or two children while maintaining the BST property.

5. The inorder(), preorder(), and postorder() functions display the tree elements in different orders. Inorder displays elements in ascending order.

6. The while loop displays a menu and allows the user to choose operations such as insert, search, delete, and traversal. The program continues until the user selects Exit.

Error table:

S.No.	Error	**Reason**	**Solution**
1	NameError	Class or variable is not defined.	Define the class or variable before using it.
2	IndentationError	Incorrect indentation	Use proper indentation with 4 spaces
3	ValueError	Invalid input entered instead of an integer.	Enter a valid integer.
4	AttributeError	Accessing attributes of a None object	Check whether the node is None.
5	RecursionError	Recursion depth exceeded.	Avoid excessively unbalanced trees
6	SyntaxError	Missing colon or bracket.	Correct the syntax.

Week 10

Aim: Write a python program to implement avl trees with its operation.

Input:

avl tree.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/avl tree.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:

    def height(self, root):
        if root is None:
            return 0
        return root.height

    def balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    # Right Rotation
    def right_rotate(self, y):
        x = y.left
        t = x.right

        x.right = y
        y.left = t

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))

        return x

    # Left Rotation
    def left_rotate(self, x):
        y = x.right
        t = y.left
```

avl tree.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/avl tree.py (3.14.0)

File Edit Format Run Options Window Help

```
        y = x.right
        t = y.left

        y.left = x
        x.right = t

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))

        return y

    # Insert
    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        b = self.balance(root)

        # LL Case
        if b > 1 and key < root.left.key:
            return self.right_rotate(root)

        # RR Case
        if b < -1 and key > root.right.key:
            return self.left_rotate(root)

        # LR Case
        if b > 1 and key > root.left.key:
```

avl tree.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/avl tree.py (3.14.0)

File Edit Format Run Options Window Help

```
        # LR Case
        if b > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RL Case
        if b < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Find smallest node
    def min_value(self, root):
        temp = root
        while temp.left is not None:
            temp = temp.left
        return temp

    # Delete
    def delete(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)

        elif key > root.key:
            root.right = self.delete(root.right, key)

        else:
            if root.left is None:
                return root.right

            elif root.right is None:
                return root.left

            temp = self.min_value(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)
```

avl tree.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/avl tree.py (3.14.0)

File Edit Format Run Options Window Help

```
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        b = self.balance(root)

        # LL Case
        if b > 1 and self.balance(root.left) >= 0:
            return self.right_rotate(root)

        # LR Case
        if b > 1 and self.balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RR Case
        if b < -1 and self.balance(root.right) <= 0:
            return self.left_rotate(root)

        # RL Case
        if b < -1 and self.balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Search
    def search(self, root, key):
        if root is None:
            return False

        if root.key == key:
            return True

        if key < root.key:
```

avl tree.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/avl tree.py (3.14.0)

File Edit Format Run Options Window Help

```
        if key < root.key:
            return self.search(root.left, key)

        return self.search(root.right, key)

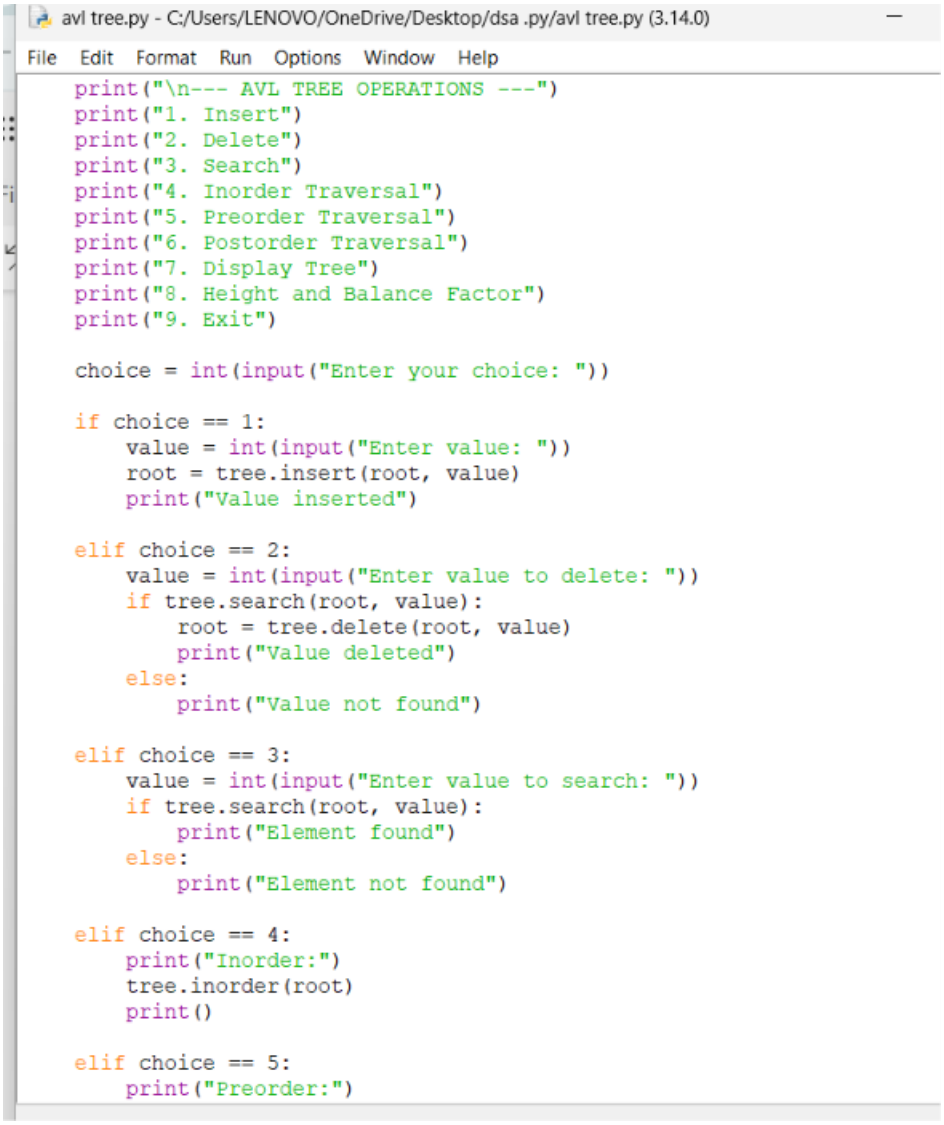
    # Inorder Traversal
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    # Preorder Traversal
    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Postorder Traversal
    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")

    # Display Height and Balance Factor
    def show_details(self, root):
        if root:
            self.show_details(root.left)
            print("Node:", root.key,
                  "Height:", root.height,
                  "Balance Factor:", self.balance(root))
            self.show_details(root.right)

    # Display Tree
    def display(self, root, level=0):
        if root:
            self.display(root.right, level + 1)
```


avl tree.py - C:/Users/LENOVO/OneDrive/Desktop/dsa .py/avl tree.py (3.14.0)
File Edit Format Run Options Window Help
print("\n--- AVL TREE OPERATIONS ---")
print("1. Insert")
print("2. Delete")
print("3. Search")
print("4. Inorder Traversal")
print("5. Preorder Traversal")
print("6. Postorder Traversal")
print("7. Display Tree")
print("8. Height and Balance Factor")
print("9. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
value = int(input("Enter value: "))
root = tree.insert(root, value)
print("Value inserted")
elif choice == 2:
value = int(input("Enter value to delete: "))
if tree.search(root, value):
root = tree.delete(root, value)
print("Value deleted")
else:
print("Value not found")
elif choice == 3:
value = int(input("Enter value to search: "))
if tree.search(root, value):
print("Element found")
else:
print("Element not found")
elif choice == 4:
print("Inorder:")
tree.inorder(root)
print()
elif choice == 5:
print("Preorder:")

Output:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
========= RESTART: C:/Users/LENOVO/OneDrive/Desktop/dsa .py/avl tree.py ========
--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Height and Balance Factor
9. Exit
Enter your choice: 1
Enter value: 12
Value inserted
--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Height and Balance Factor
9. Exit
Enter your choice: 3
Enter value to search: 12
Element found
--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Height and Balance Factor
9. Exit
Ln: 224 Col:



Explanation:

1. The Node class is used to create a node containing data and two pointers, left and right, which point to the left and right child nodes.

2. The insert() function adds a new node to the Binary Search Tree. Smaller values are placed on the left side, and larger values are placed on the right side.

3. The search() function checks whether the given element exists in the tree. It compares the element with the current node and searches the left or right subtree.

4. The delete() function removes a node from the tree. It handles nodes with no children, one child, or two children while maintaining the BST property.

5. The inorder(), preorder(), and postorder() functions display the tree elements in different orders. Inorder displays elements in ascending order.

6. The while loop displays a menu and allows the user to choose operations such as insert, search, delete, and traversal. The program continues until the user selects Exit

Error table:

Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function name	Rectified
2.	Passing a value of the wrong data type.	Rectified.

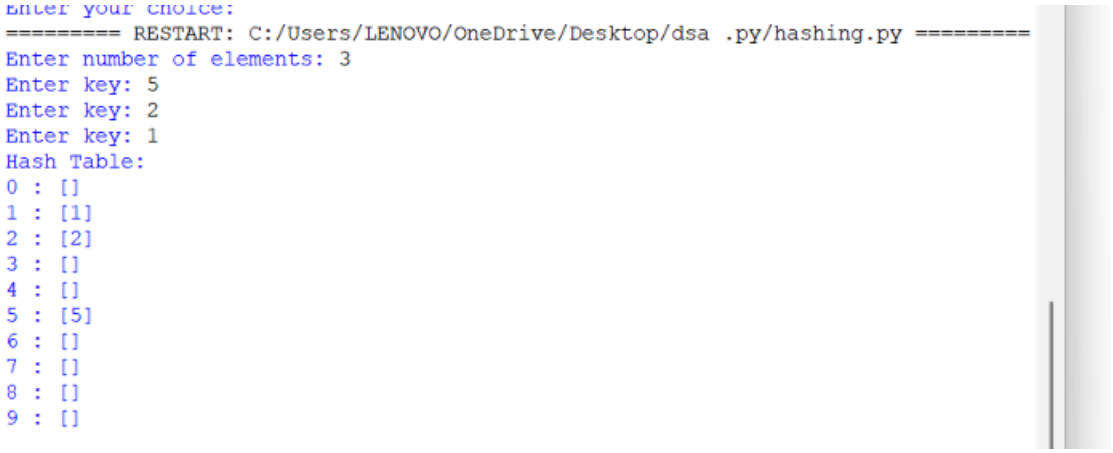
Week-11

Aim: Python programs for hashing using separate chaining, linear probing, and quadratic probing

Input:



Output:



Explanation:

- The size = 10 statement creates a hash table with 10 positions.
- The table = [[] for i in range(size)] statement creates an empty list at every position.
- The insert() function calculates the index using key % size.
- If two keys have the same index, both keys are stored in the same list.
- The display() function prints each index and its stored elements.
- The for loop accepts keys from the user and inserts them into the hash table.
- Error table:

S.no	Errors	Rectified
1.	Adding incompatible data types.	Rectified
2.	Entering a non-integer key.	Rectified.

-
-